

Module 10: Aggregate Functions

Aggregate functions perform calculations on multiple rows and return a single summarized value. They are among the most important tools in SQL for reporting, business intelligence, and data analytics.



Introduction

Imagine a company database containing thousands of employees.

Instead of viewing every employee record individually, management may ask:

- How many employees are there?
- What is the total salary expense?
- What is the average salary?
- What is the highest salary?
- What is the lowest salary?

Aggregate functions answer these questions.



Learning Objectives

After completing this module, you will be able to:

- ✓ Understand aggregate functions
 - ✓ Use COUNT()
 - ✓ Use SUM()
 - ✓ Use AVG()
 - ✓ Use MIN()
 - ✓ Use MAX()
 - ✓ Handle NULL values correctly
 - ✓ Apply aggregates in business scenarios
 - ✓ Write analytical SQL queries
-



Table of Contents

1. What are Aggregate Functions?
2. Aggregate Function Rules

3. COUNT()
4. SUM()
5. AVG()
6. MIN()
7. MAX()
8. Aggregate Functions and NULL Values
9. Multiple Aggregate Functions
10. Aggregate Functions with WHERE
11. Business Analytics Examples
12. Common Mistakes
13. Best Practices
14. Summary
15. Practice Questions

1 What are Aggregate Functions?

Aggregate functions perform calculations on a set of rows and return a single result.

Example Table

EmployeeID	EmployeeName	Salary
101	Kunal	45000
102	Rahul	50000
103	Aman	60000

Question:

```
What is the total salary?
```

Aggregate Function:

```
SELECT SUM(Salary)
FROM Employees;
```

Output:

```
155000
```

Why Aggregate Functions Matter

Used in:

- Dashboards
 - Reports
 - KPI calculations
 - Financial analysis
 - Business intelligence
-

Common Aggregate Functions

Function	Purpose
COUNT()	Counts rows
SUM()	Adds values
AVG()	Calculates average
MIN()	Smallest value
MAX()	Largest value

2 Aggregate Function Rules

Aggregate functions:

- ✓ Work on multiple rows
 - ✓ Return a single value
 - ✓ Ignore NULL values (except COUNT(*))
-

Example:

```
SELECT AVG(Salary)
FROM Employees;
```

Returns one value.

3 COUNT()

Used to count records.

COUNT(*)

Counts all rows.

Example

```
SELECT COUNT(*)  
FROM Employees;
```

Output:

```
100
```

Meaning:

```
100 rows exist
```

Example Table

EmployeeID	Name
101	Kunal
102	Rahul
103	Aman

Query

```
SELECT COUNT(*)  
FROM Employees;
```

Output:

```
3
```

COUNT(Column Name)

Counts only non-NULL values.

Example

EmployeeID	Bonus
101	5000
102	NULL
103	2000

Query

```
SELECT COUNT(Bonus)
FROM Employees;
```

Output:

```
2
```

NULL is ignored.

COUNT(DISTINCT)

Counts unique values.

Example

City
Delhi
Delhi
Mumbai
Pune

Query

```
SELECT COUNT(DISTINCT City)
FROM Employees;
```

Output:

3

Real-World Uses

- Number of customers
 - Number of orders
 - Number of products
 - Unique cities
-



SUM()

Adds numeric values.

Example Table

Salary
45000
50000
60000

Query

```
SELECT SUM(Salary)
FROM Employees;
```

Output

155000

Example

```
SELECT SUM(Quantity)
FROM Orders;
```

Real-World Uses

- Total Revenue
 - Total Sales
 - Total Expenses
 - Total Quantity Sold
-

5 AVG()

Calculates average value.

Formula

```
Total ÷ Count
```

Example

Salary
45000
50000
60000

Query

```
SELECT AVG(Salary)
FROM Employees;
```

Output

```
51666.67
```

Example

```
SELECT AVG(Marks)
FROM Students;
```

Real-World Uses

- Average Salary
 - Average Revenue
 - Average Customer Spend
 - Average Rating
-

MIN()

Returns the smallest value.

Example

Salary
45000
50000
60000

Query

```
SELECT MIN(Salary)
FROM Employees;
```

Output

```
45000
```

Works on Dates

Example

```
SELECT MIN(OrderDate)
FROM Orders;
```

Output

```
Earliest order date
```

Real-World Uses

- Lowest Salary
- Cheapest Product
- Earliest Order
- Minimum Score

MAX()

Returns largest value.

Example

```
SELECT MAX(Salary)
FROM Employees;
```

Output

```
60000
```

Works on Dates

Example

```
SELECT MAX(OrderDate)
FROM Orders;
```

Output

Latest order date

Real-World Uses

- Highest Salary
- Highest Revenue
- Latest Transaction
- Maximum Score

Aggregate Functions and NULL Values

Aggregate functions generally ignore NULL values.

Example Table

Salary

45000

NULL

60000

COUNT(*)

```
SELECT COUNT(*)
FROM Employees;
```

Output

3

Counts all rows.

COUNT(Salary)

```
SELECT COUNT(Salary)
FROM Employees;
```

Output

2

NULL ignored.

AVG(Salary)

```
SELECT AVG(Salary)
FROM Employees;
```

Output

52500

Computed using:

45000 + 60000

Only.

Important Rule

Aggregate functions ignore NULL values except:

```
COUNT(*)
```

9 Multiple Aggregate Functions

You can use multiple aggregates in one query.

Example

```
SELECT COUNT(*) AS TotalEmployees,  
       SUM(Salary) AS TotalSalary,  
       AVG(Salary) AS AverageSalary,  
       MIN(Salary) AS LowestSalary,  
       MAX(Salary) AS HighestSalary  
FROM Employees;
```

Output

Metric	Value
TotalEmployees	100
TotalSalary	5000000
AverageSalary	50000
LowestSalary	25000
HighestSalary	150000

10 Aggregate Functions with WHERE

Aggregates can be combined with filtering.

Example

```
SELECT AVG(Salary)  
FROM Employees  
WHERE Department = 'IT';
```

Meaning:

Average salary of IT employees only

Example

```
SELECT COUNT(*)  
FROM Orders  
WHERE OrderDate >= '2025-01-01';
```

Meaning:

Orders placed in 2025

1 1 Business Analytics Examples

Total Revenue

```
SELECT SUM(SalesAmount)  
FROM Sales;
```

Average Order Value

```
SELECT AVG(OrderAmount)  
FROM Orders;
```

Highest Paid Employee

```
SELECT MAX(Salary)
```

```
FROM Employees;
```

Total Customers

```
SELECT COUNT(*)  
FROM Customers;
```

Number of Unique Cities

```
SELECT COUNT(DISTINCT City)  
FROM Customers;
```

KPI Dashboard Query

```
SELECT COUNT(*) AS TotalOrders,  
       SUM(OrderAmount) AS Revenue,  
       AVG(OrderAmount) AS AvgOrderValue,  
       MAX(OrderAmount) AS HighestOrder,  
       MIN(OrderAmount) AS LowestOrder  
FROM Orders;
```

1 **2** Common Mistakes

Using SUM on Text

Bad

```
SELECT SUM(EmployeeName)
```

```
FROM Employees;
```

Error.

SUM requires numeric data.

Forgetting NULL Behavior

Incorrect assumptions can produce wrong results.

COUNT(Column) vs COUNT(*)

Example

```
COUNT(*)
```

Counts all rows.

```
COUNT(Salary)
```

Counts non-NULL salaries only.

Wrong Data Type

Using AVG on text columns.

Ignoring WHERE

Example

```
SELECT AVG(Salary)
FROM Employees;
```

May not answer:

Average IT salary

Need:

```
WHERE Department='IT'
```

1 3 Best Practices

Use Aliases

Good

```
SELECT AVG(Salary) AS AverageSalary  
FROM Employees;
```

Use COUNT(*)

For row counts.

Handle NULL Values

Understand how aggregates treat NULLs.

Filter Before Aggregating

Example

```
WHERE Department='IT'
```

Combine Multiple Aggregates

Reduces database calls.

Professional Example

```
SELECT COUNT(*) AS TotalEmployees,  
  
       AVG(Salary) AS AvgSalary,  
  
       MIN(Salary) AS LowestSalary,  
  
       MAX(Salary) AS HighestSalary  
  
FROM Employees;
```



Summary

In this module, you learned:

- ✓ COUNT()
 - ✓ SUM()
 - ✓ AVG()
 - ✓ MIN()
 - ✓ MAX()
 - ✓ NULL Handling
 - ✓ Aggregate Rules
 - ✓ Business Analytics Queries
 - ✓ KPI Calculations
-



Practice Questions

Theory

1. What is an aggregate function?
2. What does COUNT(*) do?
3. Difference between COUNT(*) and COUNT(Column)?
4. What does SUM() do?
5. What does AVG() do?
6. What does MIN() do?
7. What does MAX() do?
8. How do aggregate functions handle NULL values?
9. What is COUNT(DISTINCT)?
10. Why use aliases with aggregates?

Practical Exercises

Task 1

Find:

Total Employees

using:

COUNT(*)

Task 2

Find:

Total Salary Expense

using:

SUM()

Task 3

Find:

Average Salary

using:

AVG()

Task 4

Find:

Highest Salary

using:

MAX()

Task 5

Find:

Lowest Salary

using:

MIN()

Task 6

Count unique cities.

Task 7

Calculate average salary of IT employees only.

Challenge Project

Create an Employee Analytics Report containing:

- Total Employees
- Total Salary Expense
- Average Salary
- Highest Salary
- Lowest Salary
- Number of Unique Cities

Use a single SQL query.



Next Module

→ **Module 11: GROUP BY Clause**

Topics Covered:

- GROUP BY Basics
- Grouping Records
- Aggregates with GROUP BY
- Multiple Column Grouping
- GROUP BY vs DISTINCT
- Common Errors
- Real-World Business Reports